

Laboratorio 8 — Respuestas teóricas

Teoría de la Computación

Octubre 2026

Este documento incluye la parte (a) de los problemas 1, 2 y 3, el problema 4 y el problema 5. Las partes (b) (implementación y profiling) están en el repositorio.

Problema 1 (a)

```
1  for (i = n/2; i <= n; i++)
2      for (j = 1; j+n/2 <= n; j++)
3          for (k = 1; k <= n; k = k*2)
4              counter++;
```

Los tres ciclos son independientes entre sí (ninguno depende de la variable del ciclo de afuera), así que el total es el producto de las vueltas de cada uno.

Ciclo de i . Va de $\lfloor n/2 \rfloor$ hasta n de uno en uno:

$$\text{vueltas}_i = n - \lfloor n/2 \rfloor + 1 \approx \frac{n}{2} + 1$$

Ciclo de j . La condición $j + \lfloor n/2 \rfloor \leq n$ es lo mismo que $j \leq n - \lfloor n/2 \rfloor$, entonces:

$$\text{vueltas}_j = n - \lfloor n/2 \rfloor = \lceil n/2 \rceil \approx \frac{n}{2}$$

Ciclo de k . k toma los valores $1, 2, 4, \dots, 2^m$ mientras $2^m \leq n$, o sea $m \leq \log_2 n$:

$$\text{vueltas}_k = \lfloor \log_2 n \rfloor + 1$$

Total.

$$T(n) = (n - \lfloor n/2 \rfloor + 1) \cdot \lceil n/2 \rceil \cdot (\lfloor \log_2 n \rfloor + 1) \approx \frac{n}{2} \cdot \frac{n}{2} \cdot \log_2 n = \frac{n^2 \log_2 n}{4}$$

Quitando constantes y términos de menor orden:

$$T(n) = O(n^2 \log n) \quad (\text{de hecho } \Theta(n^2 \log n), \text{ porque la cota también es inferior}).$$

Comprobación con el programa (el contador real coincide con la fórmula):

n	fórmula	counter
10	$6 \cdot 5 \cdot 4 = 120$	120
100	$51 \cdot 50 \cdot 7 = 17\,850$	17 850
1000	$501 \cdot 500 \cdot 10 = 2\,505\,000$	2 505 000

Problema 2 (a)

```
1  if (n <= 1) return;
2  for (i = 1; i <= n; i++)
3      for (j = 1; j <= n; j++) {
4          printf("Sequence\n");
5          break;
6      }
```

A primera vista parecen dos ciclos anidados de n (lo que daría n^2), pero el **break** hace que el ciclo de j se salga en la **primera** vuelta, siempre. Entonces el ciclo interno cuesta una constante c (una comparación, un **printf** y el **break**).

- Si $n \leq 1$: solo se evalúa el **if** $\Rightarrow$ tiempo constante.
- Si $n > 1$: el ciclo de i da n vueltas y cada una cuesta c .

$$T(n) = c_0 + \sum_{i=1}^n c = c_0 + cn$$

$$T(n) = O(n) \quad (\Theta(n): \text{se imprimen exactamente } n \text{ líneas}).$$

Problema 3 (a)

```
1  for (i = 1; i <= n/3; i++)
2      for (j = 1; j <= n; j += 4)
3          printf("Sequence\n");
```

Ciclo de i .

$$\text{vueltas}_i = \lfloor n/3 \rfloor$$

Ciclo de j . j toma los valores $1, 5, 9, \dots, 1 + 4t$ mientras $1 + 4t \leq n$, o sea $t \leq (n-1)/4$:

$$\text{vueltas}_j = \left\lfloor \frac{n-1}{4} \right\rfloor + 1 = \lceil n/4 \rceil$$

Total. El ciclo de j no depende de i , entonces:

$$T(n) = \lfloor n/3 \rfloor \cdot \lceil n/4 \rceil \approx \frac{n}{3} \cdot \frac{n}{4} = \frac{n^2}{12}$$

$$T(n) = O(n^2) \quad (\Theta(n^2)).$$

Comprobación: con $n = 1000$ hay $333 \cdot 250 = 83\,250$ impresiones, que es lo que cuenta el programa. Con $n = 1$ no imprime nada porque $\lfloor 1/3 \rfloor = 0$.

Problema 4: mejor caso, caso promedio y peor caso

4.1 Búsqueda lineal

```
1 for i in 0..n-1:
2     if A[i] == x: return i
3 return -1
```

Se cuenta el número de comparaciones $A[i] == x$.

- **Mejor caso:** x está en la primera posición $\Rightarrow$ 1 comparación $\Rightarrow \Theta(1)$.
- **Peor caso:** x está en la última posición o no está $\Rightarrow n$ comparaciones $\Rightarrow \Theta(n)$.
- **Caso promedio:** suponiendo que x está y que cada posición es igual de probable ($1/n$), si está en la posición i se hacen i comparaciones:

$$E[C] = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

$\Rightarrow \Theta(n)$. Si además x puede no estar, el promedio queda entre $\frac{n+1}{2}$ y n , y sigue siendo $\Theta(n)$.

4.2 Búsqueda binaria (arreglo ordenado)

```
1 lo = 0, hi = n-1
2 while lo <= hi:
3     mid = (lo+hi)/2
4     if A[mid] == x: return mid
5     if A[mid] < x: lo = mid+1 else: hi = mid-1
```

- **Mejor caso:** x está justo en el medio $\Rightarrow$ 1 comparación $\Rightarrow \Theta(1)$.
- **Peor caso:** cada vuelta deja la mitad del rango:

$$T(n) = T(n/2) + c \quad \Rightarrow \quad T(n) = c(\lfloor \log_2 n \rfloor + 1)$$

Después de k vueltas quedan $n/2^k$ elementos; se acaba cuando $n/2^k < 1$, o sea $k \approx \log_2 n \Rightarrow \Theta(\log n)$.

- **Caso promedio:** se piensa la búsqueda como un árbol binario de decisión. Con $n = 2^k - 1$, en el nivel d hay 2^{d-1} elementos que se encuentran con d comparaciones:

$$E[C] = \frac{1}{n} \sum_{d=1}^k d \cdot 2^{d-1} = \frac{(k-1)2^k + 1}{2^k - 1} \approx k - 1 = \log_2(n+1) - 1$$

$\Rightarrow \Theta(\log n)$. Más de la mitad de los elementos está en el último nivel, por eso el promedio es casi igual al peor caso.

4.3 Quicksort

Se elige un pivote, se particiona en $O(n)$ (aprox. n comparaciones) y se ordenan las dos partes recursivamente. Si el pivote queda en la posición q :

$$T(n) = T(q) + T(n - 1 - q) + c n$$

- **Mejor caso:** el pivote siempre parte el arreglo a la mitad:

$$T(n) = 2T(n/2) + c n$$

Por el teorema maestro ($a = 2$, $b = 2$, $f(n) = n = n^{\log_2 2}$) es el caso 2. También se ve con el árbol de recursión: hay $\log_2 n$ niveles y cada nivel cuesta $c n \Rightarrow \Theta(n \log n)$.

- **Peor caso:** el pivote siempre es el mínimo o el máximo (por ejemplo, un arreglo ya ordenado con pivote igual al último elemento):

$$T(n) = T(n - 1) + c n = c(n + (n - 1) + \dots + 1) = c \frac{n(n + 1)}{2}$$

$$\Rightarrow \Theta(n^2).$$

- **Caso promedio:** suponiendo que el pivote cae en cualquier posición con probabilidad $1/n$:

$$T(n) = c n + \frac{1}{n} \sum_{q=0}^{n-1} [T(q) + T(n - 1 - q)] = c n + \frac{2}{n} \sum_{q=0}^{n-1} T(q)$$

Multiplicando por n y restando la misma ecuación para $n - 1$:

$$\begin{aligned} n T(n) - (n - 1) T(n - 1) &= c(2n - 1) + 2 T(n - 1) \\ \Rightarrow n T(n) &= (n + 1) T(n - 1) + c(2n - 1) \end{aligned}$$

Dividiendo entre $n(n + 1)$:

$$\frac{T(n)}{n + 1} = \frac{T(n - 1)}{n} + \frac{c(2n - 1)}{n(n + 1)} \leq \frac{T(n - 1)}{n} + \frac{2c}{n + 1}$$

Desenrollando la suma:

$$\frac{T(n)}{n + 1} \approx 2c \sum_{k=1}^n \frac{1}{k} = 2c H_n \approx 2c \ln n \Rightarrow T(n) \approx 2c n \ln n \approx 1.39 c n \log_2 n$$

$$\Rightarrow \Theta(n \log n).$$

Algoritmo	Mejor	Promedio	Peor
Búsqueda lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Búsqueda binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

Tabla 1: Resumen del problema 4.

Problema 5: verdadero o falso

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$. **VERDADERO**

Por definición existen constantes positivas tales que, para $n \geq \max(n_1, n_2)$:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{y} \quad d_1 h(n) \leq g(n) \leq d_2 h(n)$$

Sustituyendo g en la primera:

$$c_1 d_1 h(n) \leq f(n) \leq c_2 d_2 h(n) \Rightarrow f(n) = \Theta(h(n)) \quad (\text{transitividad})$$

Despejando h :

$$\frac{f(n)}{c_2 d_2} \leq h(n) \leq \frac{f(n)}{c_1 d_1} \Rightarrow h(n) = \Theta(f(n)) \quad (\text{simetría})$$

Las constantes $\frac{1}{c_2 d_2}$ y $\frac{1}{c_1 d_1}$ son positivas, así que se cumple la definición.

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$. **VERDADERO**

Existen $c, d > 0$ tales que, para n suficientemente grande, $f(n) \leq c g(n)$ y $g(n) \leq d h(n)$. Entonces:

$$f(n) \leq c d h(n) \Rightarrow h(n) \geq \frac{1}{cd} f(n)$$

Eso es justo la definición de $h(n) = \Omega(f(n))$ con constante $\frac{1}{cd} > 0$. (En general, $f = O(h) \iff h = \Omega(f)$.)

c) $f(n) = \Theta(n^2)$ para el programa **A(n)**. **FALSO**, en realidad es $\Theta(n^3)$

```
1 atupla = tuple(range(0, n))
2 S = set()
3 for i in range(0, n):
4     for j in range(i + 1, n):
5         S.add(atupla[i:j])
```

Es cierto que el cuerpo del ciclo se ejecuta $\frac{n(n-1)}{2}$ veces, pero el cuerpo **no** es $O(1)$:

1. `atupla[i:j]` crea una tupla **nueva** copiando $j - i$ elementos $\Rightarrow \Theta(j - i)$.
2. `S.add(...)` tiene que calcular el hash de la tupla, y el hash de una tupla en Python recorre todos sus elementos (y no se guarda en caché) $\Rightarrow \Theta(j - i)$. Todas las tuplas son distintas, así que casi nunca hay que compararlas completas.

Con $L = j - i$ (el largo de la tupla), para cada L hay $n - L$ pares (i, j) con ese largo:

$$\begin{aligned} f(n) &= \Theta(n) + \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j - i) = \sum_{L=1}^{n-1} L (n - L) \\ &= n \cdot \frac{n(n-1)}{2} - \frac{(n-1)n(2n-1)}{6} = \frac{n^3 - n}{6} \end{aligned}$$

$f(n) = \Theta(n^3)$, entonces $f(n) \neq \Theta(n^2)$ y el enunciado es falso.

La memoria también es $\Theta(n^3)$, porque el set guarda todas esas tuplas.

Comprobación experimental (Python 3.14). Si f fuera $\Theta(n^2)$, duplicar n multiplicaría el tiempo por 4; con $\Theta(n^3)$ lo multiplica por 8.

n	tiempo (s)	razón vs. $n/2$
100	0.0028	—
200	0.0290	$\times 10.3$
400	0.1894	$\times 6.5$
800	1.5941	$\times 8.4$

La razón queda alrededor de 8, lo que confirma el comportamiento cúbico.